

编译原理大作业 1

类 Rust 语言词法和语法分析工具设计与实现

2351078 李堂辉 2351837 高胜寒 2352197 冉子易

同济大学

(按学号排序)

2026 年 4 月 14 日

目录

1	概述	3
1.1	任务描述	3
1.2	实现语言与工具	3
1.3	已实现的语法规则	4
2	词法分析原理与设计	5
2.1	词法分析原理	5
2.2	状态转换图	5
2.3	标识符与关键字的识别	6
2.4	注释的处理	7
2.5	Token 数据结构	7
3	语法分析原理与设计	7
3.1	语法分析原理	7
3.2	左递归的消除	7
3.3	运算符优先级	8
3.4	语义检查：符号表与可变性	8
3.5	模块设计	9
4	详细设计	10
4.1	词法分析器详细设计	10
4.2	语法分析器详细设计	11
4.3	AST 节点设计	11

目录	2
4.4 错误处理	12
5 测试与验证	12
5.1 测试用例设计	12
5.2 词法分析结果示例	12
5.3 语法分析结果示例	13
5.4 测试结果总结	13
6 环境配置与运行指南	14
6.1 环境配置	14
6.2 编译与测试	14
7 心得与思考	14
7.1 关于 LR(1) 无法处理的情况	14
7.2 1.4 形参列表的语言	14
7.3 9.1 元组 vs 8.1 数组的产生式差异	15
8 参考资料	15

1 概述

1.1 任务描述

本项目实现了一个面向类 Rust 语言子集的前端编译分析工具，包括**词法分析器** (Lexer) 和**语法分析器** (Parser)。该工具能够：

1. 对输入的类 Rust 源代码进行词法分析，识别关键字、标识符、数值、运算符、界符等各类词法单元 (Token)；
2. 对词法分析产生的 Token 序列进行语法分析，按照给定的上下文无关语法规则构建抽象语法树 (AST)；
3. 输出完整的词法分析结果和语法分析结果 (AST 的树形打印)。

1.2 实现语言与工具

- **实现语言**：Rust
- **词法分析方法**：手写状态机扫描器 (Cursor-based Lexer)
- **语法分析方法**：手写递归下降分析器 (Recursive Descent Parser)
- **构建工具**：Cargo (Rust 标准构建系统)

1.3 已实现的语法规则

表 1: 已实现的语法规则概览

编号	规则名称	说明
0.1	变量属性	<code>mut</code> / 空
0.2	类型	<code>i32</code> , <code>&T</code> , <code>&mut T</code> , <code>[T;N]</code> , <code>(T1,T2)</code>
0.3	左值	标识符、解引用、数组下标、元组下标
1.1	基础程序	程序 $\rightarrow$ 声明串
1.2	语句	语句串 $\rightarrow$ 语句语句串
1.3	返回语句	<code>return</code> [表达式] ;
1.4	函数输入	形参列表
1.5	函数输出	函数返回类型
2.0	变量声明	<code>let</code> [mut] <code>id</code> [:type]
2.1	变量声明语句	<code>let</code> 声明 ;
2.2	赋值语句	左值 = 表达式 ;
2.3	变量声明赋值	<code>let</code> 声明 = 表达式 ;
3.1	基本表达式	整数、标识符、括号表达式
3.2	比较运算	<code><</code> , <code><=</code> , <code>></code> , <code>>=</code> , <code>==</code> , <code>!=</code>
3.3	加减运算	<code>+</code> , <code>-</code>
3.4	乘除运算	<code>*</code> , <code>/</code>
3.5	函数调用	<code>id(args)</code>
4.1	if 选择	<code>if</code> 表达式块
4.2	if-else	增加 <code>else</code> 分支
4.3	if-else if	增加 <code>else if</code> 分支
5.0	循环语句	<code>while</code> / <code>for</code> / <code>loop</code>
5.1	while 循环	<code>while</code> 表达式块
5.2	for 循环	<code>for</code> 变量 <code>in</code> 可迭代块
5.3	loop 循环	<code>loop</code> 块
5.4	break/continue	循环控制
6.1–6.4	引用与借用	不可变/可变引用、解引用
7.0–7.4	函数表达式块	表达式块、选择表达式、循环表达式
8.1–8.3	数组	数组类型、数组字面量、数组下标
9.1–9.3	元组	元组类型、元组字面量、元组下标

2 词法分析原理与设计

2.1 词法分析原理

词法分析器的任务是将源代码字符流转换为有意义的词法单元 (Token) 序列。本项目采用**手写状态机**方法，逐字符扫描输入，根据当前字符和后续字符的模式，确定 Token 的类型和边界。

核心思想如下：

1. 维护一个**游标** (cursor)，指向当前待处理的字符位置；
2. 跳过空白字符和注释；
3. 根据当前字符判断进入何种 Token 的识别状态；
4. 对于需要前瞻的情况（如 `=` vs `==`，`-` vs `->`，`.` vs `..`），查看下一个字符来区分；
5. 对于标识符和关键字，先统一按标识符规则读取完整单词，再查表判断是否为关键字。

2.2 状态转换图

图 1 展示了词法分析器的主状态转换图。

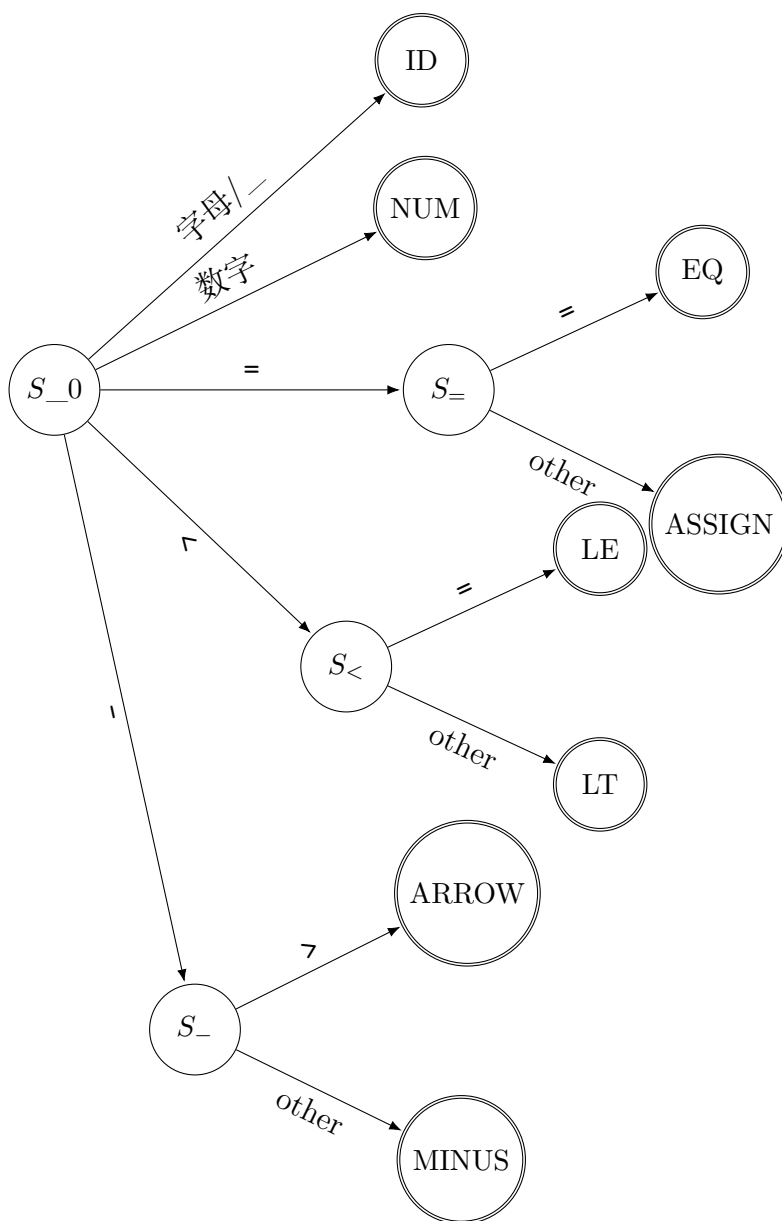


图 1: 词法分析器主状态转换图 (部分)

2.3 标识符与关键字的识别

标识符的正则定义为:

$$\text{Ident} = (\text{字母} \mid _)(\text{字母} \mid \text{数字} \mid _)^*$$

在识别出一个完整的标识符字符串后, 通过关键字表进行匹配。关键字表包含以下 13 个关键字:

i32, let, if, else, while, return, mut, fn, for, in, loop, break, continue

例如, 输入 `if123` 将被识别为**标识符** `if123` (因为字符后面跟着数字), 而 `if=123` 则被识别为三个 Token: $\langle \text{关键字, if} \rangle$ 、 $\langle \text{赋值号, =} \rangle$ 、 $\langle \text{整数, 123} \rangle$ 。

2.4 注释的处理

- `//`: 行注释, 跳过直到换行符;
- `/* ... */`: 块注释, 跳过直到遇到 `*/`, 若未闭合则报错。

2.5 Token 数据结构

每个 Token 包含三个字段:

- **kind**: Token 的类型 (枚举), 如 `Ident`, `IntLiteral`, `Plus` 等;
- **line**: 该 Token 在源码中的行号;
- **col**: 该 Token 在源码中的列号。

输出格式为: (类别, 值, 行号: 列号), 例如 (关键字, `fn`, 3:1)。

3 语法分析原理与设计

3.1 语法分析原理

本项目采用**递归下降分析法** (Recursive Descent Parsing), 这是一种自顶向下的语法分析方法。其核心思想为:

- 为文法中的每一个非终结符编写一个对应的解析函数;
- 每个函数根据当前 Token (前瞻符号) 决定选择哪条产生式进行展开;
- 对于终结符, 调用 `expect()` 函数进行匹配和消耗;
- 对于非终结符, 递归调用对应的解析函数。

3.2 左递归的消除

原始文法中的表达式规则存在**左递归**, 例如:

$$\langle \text{加减表达式} \rangle \rightarrow \langle \text{加减表达式} \rangle \langle \text{加减运算符} \rangle \langle \text{项} \rangle \quad (1)$$

$$\langle \text{项} \rangle \rightarrow \langle \text{项} \rangle \langle \text{乘除运算符} \rangle \langle \text{因子} \rangle \quad (2)$$

递归下降分析器无法直接处理左递归文法 (会导致无限递归)。解决方法是将左递归改写为**循环迭代**:

Listing 1: 左递归消除——以加减表达式为例

```

1  fn parse_add_sub(&mut self) -> Result<Expr, String> {
2      let mut left = self.parse_mul_div()?;
3      loop {
4          let op = match self.peek() {
5              TokenKind::Plus => BinOp::Add,
6              TokenKind::Minus => BinOp::Sub,
7              _ => break,
8          };
9          self.advance();
10         let right = self.parse_mul_div()?;
11         left = Expr::BinaryOp(Box::new(left), op, Box::new(right)
12             );
13     }
14     Ok(left)
15 }

```

这等价于文法改写：

$$\langle \text{加减表达式} \rangle \rightarrow \langle \text{项} \rangle \langle \text{加减表达式}' \rangle \quad (3)$$

$$\langle \text{加减表达式}' \rangle \rightarrow \langle \text{加减运算符} \rangle \langle \text{项} \rangle \langle \text{加减表达式}' \rangle \mid \varepsilon \quad (4)$$

3.3 运算符优先级

通过递归下降函数的调用层级自然地实现了运算符优先级：

表 2: 运算符优先级（从低到高）

优先级	运算符	解析函数
最低	..	parse_range_expr
	== != < <= > >=	parse_comparison
	+ -	parse_add_sub
	* /	parse_mul_div
	* & &mut (一元)	parse_unary
	[] .N (后缀)	parse_postfix
	字面量、标识符、括号	parse_primary
最高		

3.4 语义检查：符号表与可变性

为了满足 PPT 中关于“赋值语句检测”和“不可变变量赋值报错”的要求，语法分析器在解析过程中维护了一个简单的**符号表** (Symbol Table)：

- **作用域管理**：使用嵌套的 HashMap (栈式结构) 处理函数体、if 块、while 块及 for 块的局部作用域。进入块时 `push_scope`，离开时 `pop_scope`。
- **增强的可变性校验**：
 - 在解析 Assign (赋值语句) 时，递归检查左值是否为“可写左值”。
 - 支持对标识符的可变性检查 (`let mut`)。
 - 支持对可变引用解引用的检查 (如 `*ptr = 10` 仅在 `ptr` 为 `&mut i32` 时合法)。
- **严格类型校验与迭代检查**：
 - 在变量声明并赋值或普通赋值时，校验两侧类型是否完全一致，不一致则报错。
 - 对于 for 循环，严格限制 `in` 之后的表达式类型必须为 Range 或 Array。

3.5 模块设计

整个系统分为四个模块：

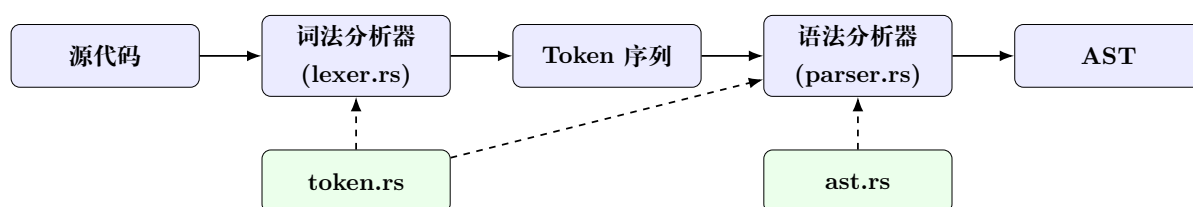


图 2: 系统模块架构图

- **token.rs**：定义 TokenKind 枚举和 Token 结构体。
- **lexer.rs**：词法分析器，将字符流转换为 Token 流。
- **ast.rs**：定义 AST 节点 (Program, FunctionDecl, Statement, Expr 等) 及其打印函数。
- **parser.rs**：递归下降语法分析器，将 Token 流转换为 AST。
- **main.rs**：程序入口，协调各模块并输出结果。

4 详细设计

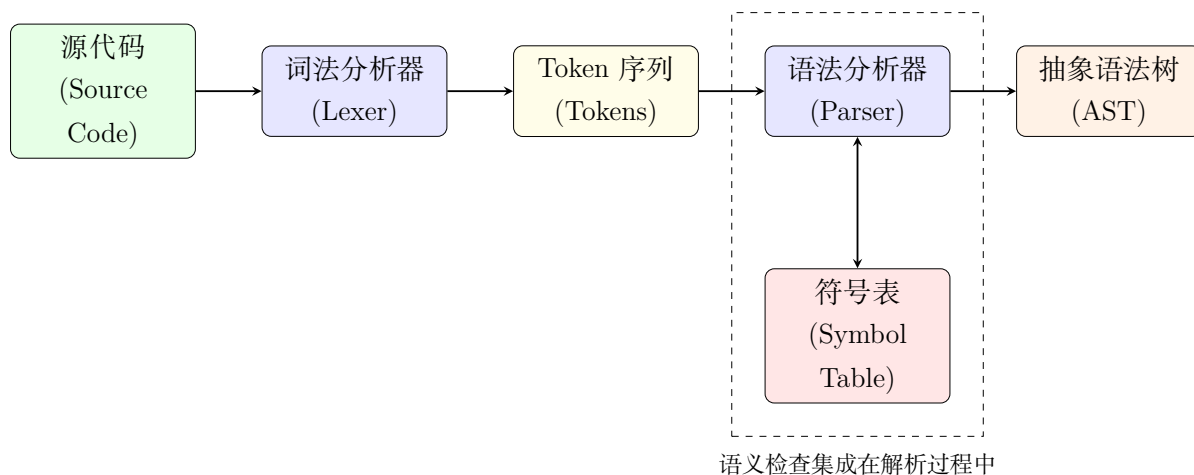


图 3: 系统模块架构图

4.1 词法分析器详细设计

Lexer 结构体包含以下字段：

```

1 pub struct Lexer {
2     input: Vec<char>,
3     pos: usize,
4     line: usize,
5     col: usize,
6 }
  
```

核心方法 `tokenize()` 的流程：

1. 调用 `skip_whitespace()` 跳过空白；
2. 检查文件结束或结束符 `#`；
3. 读取一个字符并 `advance()`；
4. 根据字符类型进入对应分支：
 - 字母或下划线 $\rightarrow$ `read_ident_or_keyword()`
 - 数字 $\rightarrow$ `read_number()`
 - 运算符/界符/分隔符 $\rightarrow$ 直接构造 Token，必要时前瞻一个字符
 - `/` $\rightarrow$ 检查注释 (`//` 或 `/* */`) 或除号
5. 将 Token 放入列表，循环直至结束。

4.2 语法分析器详细设计

Parser 结构体包含以下字段：

```
1 pub struct Parser {
2     tokens: Vec<Token>,
3     pos: usize,
4     /// 作用域栈：存储（是否可变，类型）
5     scopes: Vec<std::collections::HashMap<String, (bool, Type)>>,
6 }
```

主要解析函数及对应文法规则：

表 3: 解析函数与文法规则的对应关系

函数名	对应规则
parse_program	Program $\rightarrow$ 声明串
parse_function_decl	函数声明 $\rightarrow$ fn ID(形参) [-> 类型] 块
parse_param_list	形参列表 (1.4)
parse_type	类型 (0.2, 6.2, 6.3, 8.1, 9.1)
parse_block	语句块 $\rightarrow$ { 语句串 }
parse_statement	语句 (1.2, 1.3, 2.1–2.3, 4.1, 5.0–5.4)
parse_if_stmt	if 语句 (4.1–4.3)
parse_expression	表达式入口
parse_comparison	比较表达式 (3.2)
parse_add_sub	加减表达式 (3.3)
parse_mul_div	乘除表达式 (3.4)
parse_unary	一元表达式 (6.2–6.4)
parse_postfix	后缀表达式 (8.3, 9.3)
parse_primary	因子 (3.1, 3.5, 7.0–7.4, 8.2, 9.2)

4.3 AST 节点设计

AST 的顶层结构为 Program，包含多个 Declaration（目前仅支持函数声明）。函数声明包含函数名、参数列表、可选返回类型和函数体（Block）。

语句类型使用 Rust 枚举表示，包含：Empty, ExprStmt, Return, VarDecl, Assign, VarDeclAssign, If, While, For, Loop, Break, Continue。

表达式类型同样使用枚举，包含：IntLiteral, Ident, BinaryOp, UnaryDeref, Ref, MutRef, Call, Index, TupleIndex, ArrayLiteral, TupleLiteral, Range, ExprBlock, IfExpr, LoopExpr, Paren。

4.4 错误处理

分析过程中遇到错误时，返回包含行号、列号和错误描述的字符串。例如：

- 词法错误：3:5：非法字符'\$'
- 语法错误：7:10：期望';'，实际为'}'

5 测试与验证

5.1 测试用例设计

测试用例直接来自 PPT 中给出的示例程序，覆盖了从规则 1.1 到 9.3 的所有语法结构。内置测试包含 28 个函数定义，涵盖以下场景：

- **基础结构**：空函数体、空语句、返回语句
- **函数定义**：带参数、带返回类型
- **变量操作**：声明、赋值、声明并赋值
- **表达式**：括号嵌套、四则运算、比较运算、函数调用
- **控制流**：if/else/else if、while/for/loop、break/continue
- **高级特性**：引用/解引用、数组、元组
- **语义检查**：对不可变变量赋值报错（规则 6.1）、非迭代对象循环报错（规则 5.2 扩展）

5.2 词法分析结果示例

对于输入：

```
1 fn program_4_2(a:i32) -> i32 {  
2     if a>0 {  
3         return 1;  
4     } else {  
5         return 0;  
6     }  
7 }
```

词法分析输出（部分）：

```
1 (关键字, fn, 1:1)
2 (标识符, program\_4\_2, 1:4)
3 (界符, (, 1:15)
4 (标识符, a, 1:16)
5 (分隔符, :, 1:17)
6 (关键字, i32, 1:18)
7 (界符, ), 1:21)
8 (特殊符号, ->, 1:23)
9 (关键字, i32, 1:26)
10 (界符, {, 1:30)
11 (关键字, if, 2:5)
12 (标识符, a, 2:8)
13 (算符, >, 2:9)
14 (整数, 0, 2:10)
15 (界符, {, 2:12)
16 ...
```

5.3 语法分析结果示例

对应的 AST 输出:

```
1 FunctionDecl: program\_4\_2(a: i32) -> i32
2   Block
3     If
4       BinOp(>)
5         Ident(a)
6         Int(0)
7       Block
8         Return
9         Int(1)
10    Else
11      Block
12        Return
13        Int(0)
```

5.4 测试结果总结

所有 28 个测试函数均成功通过词法分析和语法分析, 程序输出“语法分析成功! ”。这验证了词法分析器和语法分析器对所有必需规则 (0.1–5.1) 及扩展规则 (5.2–9.3) 的正确实现。

6 环境配置与运行指南

6.1 环境配置

本项目使用 Rust 编写，编译和运行需要安装 **Rust 工具链** (1.70+ 版本推荐)。安装方法：

```
1 curl --proto '=https' --tlsv1.2 -sSf https://sh.rustup.rs | sh
```

6.2 编译与测试

在项目根目录 (2)rust_parser/ 下：

1. **编译**：运行 `cargo build --release`。
2. **运行内置测试**：运行 `cargo run --release`。
3. **测试完整用例**：运行 `cargo run --release -- tests/test_all.rs.txt`。
4. **测试语义错误拦截**：运行 `cargo run --release -- tests/test_errors.rs.txt`，预期会输出由于违反语义检查而导致的错误。

7 心得与思考

7.1 关于 LR(1) 无法处理的情况

PPT 中提到“本规则中存在极个别的羁绊会使得 LR(1) 无法处理”。一个典型的例子是**表达式块**（规则 7.0/7.1）与**普通语句块**之间的歧义。当解析器遇到 `{` 时，无法仅通过有限的前瞻确定这是一个语句块还是一个可以作为表达式的函数表达式块。

在递归下降分析器中，我们可以通过上下文信息（调用者是语句位置还是表达式位置）来区分这种情况，但在 LR(1) 分析器中，这种依赖上下文的消歧较难实现。

7.2 1.4 形参列表的语言

形参列表 $\langle \text{形参列表} \rangle \rightarrow \langle \text{形参} \rangle | \langle \text{形参} \rangle , \langle \text{形参列表} \rangle$ 可以识别**至少一个形参**的列表，产生式描述了以逗号分隔的形参序列。结合 $\langle \text{形参列表} \rangle \rightarrow \text{空}$ 的规则，完整支持零个或多个形参的情况。从语言层面看，这是一个正则语言，可用正则表达式描述为 $(\text{形参}(, \text{形参})^*)?$ 。

7.3 9.1 元组 vs 8.1 数组的产生式差异

元组类型比数组类型多了几条产生式，原因在于：

1. **数组**的所有元素类型相同，因此类型声明只需 $[T; N]$ ，一个类型加一个长度即可；
2. **元组**的每个位置可以有不同的类型，需要显式列出每个元素的类型，因此需要 $\langle \text{类型列表} \rangle$ 相关的产生式。
3. 此外，元组还需要区分 (T) 和 $(T,)$ 的情况——前者是括号括起来的类型，后者是单元素元组。

8 参考资料

1. 陈火旺. 程序设计语言编译原理（第 3 版）. 国防工业出版社.
2. Alfred V. Aho et al. Compilers: Principles, Techniques and Tools (Second Edition). 赵建华等译. 机械工业出版社.
3. The Rust Reference. <https://doc.rust-lang.org/reference/>
4. Rust By Example. <https://doc.rust-lang.org/rust-by-example/>
5. Thorsten Ball. *Writing An Interpreter In Go*. 2016.